



The American University in Cairo
CSCE 370102 - Spring 2026

Design Specification Document

Table of Contents

1.1 Purpose of the Document.....	1
1.2 Scope of Design.....	1
1.3 Definitions, Acronyms, and Abbreviations.....	1
1.4 References.....	1
2.1 Architectural Overview.....	1
2.2 Technology Stack.....	1
2.3 System Deployment Architecture.....	1
3.1 Module Design.....	1
3.2 Component Design.....	1
3.3 API Design.....	1
3.4 Database Design.....	1
3.5 Data Flow & Sequence Design.....	1
4.1 Performance Design.....	1
4.2 Security Design.....	1
4.3 Reliability & Availability.....	1
4.4 Compliance & Legal Considerations.....	1
5.1 UI Architecture.....	1
5.2 Wireframes / Mockups.....	1
5.3 Navigation Flow.....	1

6. Business Rules Implementation.....	2
7. Risks & Mitigation Strategy.....	2
8. Testing & Verification Strategy.....	2
9. Appendix.....	2

1. Introduction

1.1 Purpose of the Document

This document provides a detailed technical design of the Hirely system, describing its architecture, components, data structures, interfaces, and operational constraints. It serves as a blueprint for development, ensuring alignment between system requirements and implementation.

This document reflects the **current MVP implementation**, incorporating design decisions made during development.

1.2 Scope of Design

The design covers:

- System architecture and modules
- API and data persistence design
- User interface structure
- Security and performance considerations
- Deployment and scalability approach

The system focuses on enabling efficient hiring workflows for students, fresh graduates, and employers.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
API	Application Programming Interface
JWT	JSON Web Token
RBAC	Role-Based Access Control
UI	User Interface

DB	Database
MVP	Minimum Viable Product
REST	Representational State Transfer

1.4 References

- CSCE 370102 Software Engineering course materials
- Agile Scrum Sprint guidelines

2. System Architecture

2.1 Architectural Overview

Hirely follows a **Modular Monolithic Layered Architecture** consisting of:

- Presentation Layer (Frontend)
- Application Layer (Backend API)
- Data Layer (File-based persistence)
- External Services Layer

The system uses role-based access and layered security principles.

2.2 Technology Stack

Layer	Technology
Frontend	Next.js (React) + Tailwind CSS
Backend	Node.js + Express.js

Data Persistence	JSON files (file-based storage)
Authentication	Simplified / session-based (MVP)
File Storage	Local backend storage
Hosting	Local / Render / Railway (optional)

Note: Relational databases and cloud storage were intentionally excluded to support MVP constraints.

2.3 System Deployment Architecture (Updated)

- Frontend runs as a Next.js application
- Backend runs as a Node.js Express server
- Data is persisted using JSON files stored on the backend
- Uploaded CVs are stored locally and served via static routes

This deployment approach:

- Reduces infrastructure complexity
- Eliminates external service dependencies
- Supports local development and academic submission

3. Detailed Design

3.1 Module Design (Updated)

Modules include:

- Authentication Module
- User Management Module
- Job Management Module
- **Application Module (Apply, Track, Review)**
- File Handling Module (CV upload & download)

- Admin Module (future scope)

3.2 Component Design (Updated)

Key Components:

- **Auth Controller** – handles login and role checks
- **Job Controller** – manages job postings and updates
- **Application Controller** – handles job applications and status updates
- **DataStore Utility** – manages JSON read/write operations
- **File Upload Handler** – manages CV uploads and static access

3.3 API Design (Updated)

The backend exposes RESTful endpoints using query-based filtering:

- POST /api/applications – apply for job
- GET /api/applications?employerId=... – view applicants
- GET /api/applications?studentId=... – track applications
- PATCH /api/applications/:id/status – shortlist/reject

API design prioritizes **simplicity and extensibility**.

3.4 Database Design (Updated)

Persistence Approach:

- File-based JSON storage (no relational database)

Core Data Files:

- users.json
- jobs.json
- applications.json
- skills.json

Design Characteristics:

- Logical relationships maintained using IDs
- No joins; filtering performed in application logic
- Suitable for MVP-scale data

3.5 Data Flow & Sequence Design (Updated)

Example Flow: Job Application

1. Student logs in
2. Student applies for a job
3. Application stored in `applications.json`
4. Employer retrieves applications via filtered API
5. Employer updates application status
6. Student views updated status in Track Applications page

4. Non-Functional Design

4.1 Performance Design

- Lightweight JSON read/write operations
- Minimal payload sizes
- Client-side filtering where appropriate

4.2 Security Design

Security Model: Layered security

- Frontend: input validation
- Backend: role-based access checks
- File access: static serving without exposing filesystem paths

4.3 Reliability & Availability

- Stateless backend design
- Deterministic JSON persistence
- Graceful error handling and logging

4.4 Compliance & Legal Considerations

- User data stored locally for MVP
- Passwords hashed using secure algorithms
- Access restricted by role
- No third-party data sharing

5. User Interface Design

5.1 UI Architecture

- Component-based React UI
- Role-based dashboards:
 - Student Dashboard
 - Employer Dashboard

5.2 Wireframes / Mockups

On demo

5.3 Navigation Flow (Updated)

Student: Login → Profile → Browse Jobs → Apply → Track Applications

Employer: Login → Dashboard → Post Job → View Applicants → Shortlist/Reject

6. Business Rules Implementation (Updated)

- Users must be authenticated to perform actions
- Students cannot post jobs
- Employers cannot apply for jobs
- Employers can only manage their own job postings
- Duplicate applications are not allowed

7. Risks & Mitigation Strategy (Updated)

Risk	Mitigation
Time constraints	MVP prioritization
Data inconsistency	Centralized DataStore utility

Scalability limits	Deferred to post-MVP
Security complexity	Simplified auth for MVP

8. Testing & Verification Strategy (Updated)

Testing Types:

- Unit testing (controllers, utilities)
- Integration testing (API + persistence)
- UI testing
- User acceptance testing

Verification:

- Validate features against SRS
- Confirm persistence across restarts
- Verify role-based access